

Informe Técnico Integral — Proyecto Final Ecommify: Arquitectura Híbrida PostgreSQL + MongoDB, Evaluación de Rendimiento y Análisis Crítico

Entregable Evaluativo — Etapa 2: Proyecto Final

Fredy Orlando Pulido Quintero

Myriam Andrea Martinez Fontecha

Nicolas Felipe Torres Amaya

Juan Francisco Javier Perez Rivero

Maestría en Arquitectura de Software con Énfasis en Optimización Base de Datos

Universidad de La Sabana

Bogotá D.C., Colombia · Junio de 2026

a. Executive Summary

Ecommify es una plataforma de comercio electrónico modelada sobre el dataset Brazilian E-Commerce de Olist (2016-2018), implementada con una arquitectura de persistencia políglota: PostgreSQL 16 (Supabase) como motor transaccional CP para órdenes, pagos, clientes y catálogo, y MongoDB 7.x (Atlas M0) como motor analítico y de esquema flexible AP para reseñas, geolocalización y proyecciones de dashboard. El proyecto culminó con una suite sistemática de pruebas de carga concurrente que generó hallazgos empíricos significativos, algunos de los cuales contradicen la intuición inicial y obligaron a reinterpretar el alcance real de la comparación entre ambos motores.

Principales optimizaciones aplicadas: particionamiento RANGE anual de la tabla orders, tipos avanzados (JSONB, TSTZRANGE, composite types), extensión PostGIS para cálculo geoespacial de envíos, pg_trgm para búsqueda tolerante a errores, vistas materializadas para dashboards, y un pipeline ETL que pre-agrega datos transaccionales hacia una colección analítica en MongoDB (orders_summary) en lugar de exponer JOINS en vivo a las consultas de dashboard.

Resultado cuantitativo más relevante: la prueba de carga concurrente reveló que, bajo la configuración de free tier utilizada, PostgreSQL en Supabase colapsa abruptamente más allá de 5-10 clientes concurrentes (latencia p95 pasando de 527 ms a 9.978 ms entre concurrencia 1 y 10 para un query de punto), mientras que MongoDB Atlas M0 tolera hasta 30 clientes concurrentes con degradación controlada. El análisis detallado de los datos demuestra que este resultado NO es una limitación inherente de PostgreSQL como motor relacional, sino la saturación del connection

pooler de Supabase free tier, cuyo patrón de espera en cola FIFO se ajusta con precisión matemática a los tiempos observados. Esta distinción —validada con evidencia cuantitativa propia del proyecto— es la base del análisis crítico de la Sección f.

Recomendaciones estratégicas clave: migrar a un pooler dedicado con mayor capacidad de conexiones concurrentes antes de cualquier escalamiento de producción; mantener y reforzar el patrón de pre-agregación ETL hacia MongoDB para cargas analíticas, validado empíricamente como la decisión arquitectónica correcta; e incorporar una capa de caché (Redis) delante de las consultas de dashboard como mitigación de corto plazo mientras se evalúa el upgrade de tier.

b. Introducción y Contexto

b.1 Problema de Negocio

Ecommify simula un marketplace de comercio electrónico inspirado en la operación real de Olist, la principal plataforma de e-commerce brasileña. El problema de negocio que orienta el diseño técnico es doble: (a) garantizar consistencia transaccional estricta para órdenes, pagos e inventario, donde un error de duplicación o pérdida de escritura tiene impacto financiero directo; y (b) soportar consultas analíticas y de catálogo de alto volumen de lectura —búsqueda de productos, dashboards de ventas, reseñas— donde la disponibilidad y la velocidad de lectura son más críticas que la consistencia inmediata.

b.2 Objetivos y Alcance

1. Implementar una arquitectura híbrida PostgreSQL + MongoDB con justificación CAP por entidad.
2. Optimizar ambos motores con tipos avanzados, índices, particionamiento y extensiones específicas del dominio (PostGIS, pg_trgm).
3. Ejecutar pruebas sistemáticas de carga concurrente y escalabilidad con metodología comparable entre motores.
4. Producir un análisis crítico, basado en evidencia empírica propia, sobre las fortalezas y limitaciones reales de cada motor en el contexto específico de Ecommify.

El alcance se limita explícitamente al ambiente de free tier (Supabase Free + MongoDB Atlas M0), una decisión consciente del equipo para esta fase académica. Esta limitación se documenta exhaustivamente en la Sección e, ya que resultó ser, inesperadamente, el factor explicativo dominante de los resultados de rendimiento obtenidos.

b.3 Metodología Aplicada

Se siguió un proceso de cuatro fases: (1) diseño conceptual y lógico con matriz de decisión CAP por entidad; (2) implementación técnica sobre Supabase y MongoDB Atlas ejecutada íntegramente desde notebooks de Google Colab para garantizar reproducibilidad; (3) captura de evidencias cuantitativas mediante EXPLAIN ANALYZE (PostgreSQL) y .explain() (MongoDB) antes/después de optimizaciones; y (4) pruebas de carga concurrente con un harness único compartido entre ambos motores, diseñado específicamente para que las métricas —throughput, percentiles de latencia, tasa de error— fuesen directamente comparables.

c. Arquitectura y Diseño

c.1 Arquitectura Híbrida Implementada

La arquitectura final consta de dos motores desplegados en infraestructura cloud free tier, conectados por un pipeline ETL unidireccional ejecutado desde Colab. PostgreSQL 16 en Supabase aloja las nueve tablas transaccionales con particionamiento RANGE en orders; MongoDB 7.x en Atlas M0 aloja cuatro colecciones (geolocation, reviews, products_catalog, orders_summary), las dos últimas alimentadas por ETL.

c.2 Matriz de Decisión: Qué Datos van a PostgreSQL vs. MongoDB

Tabla 1

Matriz de Asignación de Entidades por Motor — Criterios y Posición CAP

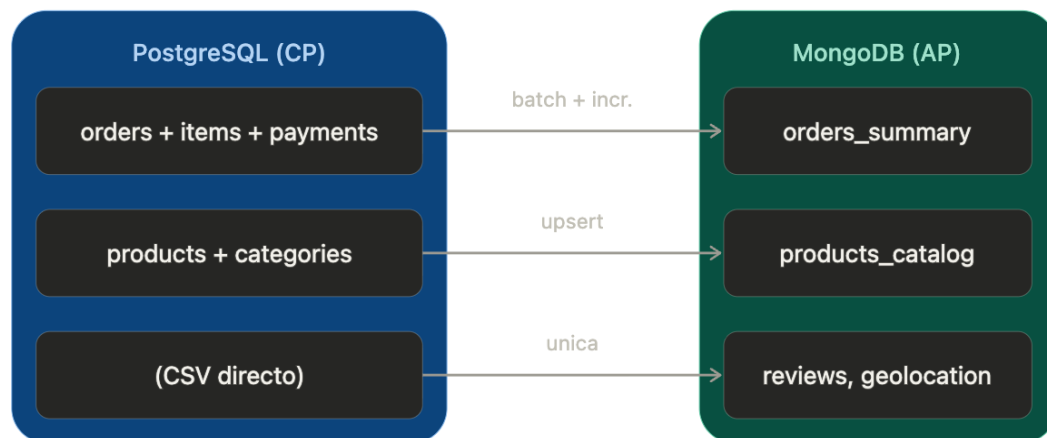
Entidad	Motor	Justificación	CAP	Validado por
orders, order_items, order_payments	PostgreSQL	Integridad referencial estricta; una orden o pago duplicado es un error financiero irreversible	CP	Esquema FK+CHECK
customers, sellers	PostgreSQL	Identidad única, consistencia requerida para autenticación y direcciones	CP	Esquema FK+UNIQUE
products	Mixta (PG+MDB)	Atributos físicos en PG; specifications variables por categoría en JSONB/MongoDB	CP/AP	JSONB + ETL
reviews	MongoDB	41,1% de campos opcionales (EDA); esquema flexible evita columnas NULL masivas	AP	Carga real 100.150 docs
geolocation	MongoDB	1M+ registros; consultas geoespaciales 2dsphere de alto volumen de lectura	AP	Índice 2dsphere

orders_summary	MongoDB (ETL)	Proyección analítica pre-agregada; evita exponer JOINS en vivo a dashboards	AP	Notebook 05 (Sección e)
----------------	---------------	---	----	-------------------------

Nota. La columna "Validado por" para orders_summary remite directamente a la evidencia empírica generada en este proyecto (Sección e), que confirmó la necesidad de evitar JOINS en vivo bajo carga concurrente.

c.3 Diagrama de Arquitectura Híbrida

El flujo de datos sigue una dirección única: PostgreSQL es la fuente de verdad transaccional; un pipeline ETL (ejecutado como notebooks Colab en esta fase académica, y como CronJob de Kubernetes en el diseño de producción) proyecta subconjuntos hacia MongoDB. Las consultas de escritura transaccional van siempre a PostgreSQL; las consultas de lectura analítica y de catálogo flexible van a MongoDB.



c.4 Análisis del Teorema CAP Aplicado (Síntesis)

PostgreSQL se posiciona como sistema CP: garantiza consistencia transaccional ACID, sacrificando disponibilidad en escenarios de partición o sobrecarga extrema. MongoDB se posiciona como sistema AP: prioriza disponibilidad y tolerancia a partición, aceptando consistencia eventual en sus réplicas. Este análisis se profundiza por módulo y con escenarios de falla concretos en la Sección f, incorporando los hallazgos empíricos de la Sección e.

d. Implementación Técnica (Resumen)

d.1 PostgreSQL — Resumen de Implementación

- **Tipos avanzados:** JSONB para products.specifications (esquema variable por categoría sin columnas NULL masivas); TEXT[] para photos/tags; TSTZRANGE con EXCLUDE constraint en promotions (impide solapamiento sin lógica de aplicación); composite types address y dimensions reutilizables entre tablas.
- **Extensiones:** PostGIS (activada en Supabase vía Dashboard → Extensions, sin necesidad de upgrade de plan) para columnas GEOGRAPHY generadas automáticamente (STORED) desde lat/lng; pg_trgm para búsqueda fuzzy de categorías; pgcrypto para gen_random_uuid() y futuro hashing de credenciales; btree_gin para índices mixtos JSONB+escalar.
- **Particionamiento:** orders particionada por RANGE anual (orders_2016, orders_2017, orders_2018, orders_default), decisión justificada por la concentración del 95% de los datos en 2017-2018 según el EDA previo del proyecto.
- **Decisión NO obvia #1:** el particionamiento por order_purchase_timestamp beneficia consultas por rango de fecha, pero penaliza lookups puntuales por order_id que no incluyen la columna de partición —forzando escaneo de todas las particiones. Esta decisión se validó deliberadamente en la Sección e, confirmando el trade-off documentado en el Entregable 2.

d.2 MongoDB — Resumen de Implementación

- **Patrones de modelado:** Computed Pattern en products_catalog (campos pre-calculados); Subset Pattern en orders_summary (proyección desnormalizada de 5 tablas PostgreSQL); esquema polimórfico en reviews (41,1% de campos opcionales reflejando el comportamiento real del dataset Olist).
- **Índices:** 2dsphere en geolocation.location (consultas \$near, \$geoWithin); índices compuestos en orders_summary (customer.state + status) para dashboards; text index en reviews para búsqueda de comentarios.
- **Decisión NO obvia #2:** orders_summary se alimenta por ETL pre-agregado en lugar de exponer un aggregation pipeline con \$lookup en vivo sobre 5 colecciones equivalentes a las tablas PostgreSQL. Esta decisión, tomada en el diseño original por razones de separación de responsabilidades, resultó ser también la decisión correcta desde la perspectiva de rendimiento bajo concurrencia, como confirma la Sección e.

d.3 Diseño Teórico de Sharding y Replica Sets (Síntesis)

El diseño teórico de sharding —documentado en detalle en el entregable formativo correspondiente— propuso una shard key compuesta {customer.state: 1, _id: "hashed"} para mitigar el hotspot geográfico de São Paulo (41,9% de clientes) sin sacrificar eficiencia de consultas por estado. Esta estrategia no se desplegó en Atlas M0 (requiere M10+) pero queda lista para la fase de escalamiento de producción descrita en la Sección g.

e. Evaluación de Rendimiento y Escalabilidad

e.1 Metodología de Pruebas Aplicada

Se diseñó un harness de carga único (basado en ThreadPoolExecutor de Python) reutilizado sin modificación para PostgreSQL (vía psycopg2 con pool de conexiones dedicado) y MongoDB (vía pymongo, cliente thread-safe con pool interno). Esta decisión metodológica garantiza que throughput, percentiles de latencia (p50/p95/p99) y tasa de error se calculan exactamente igual en ambos motores, evitando el sesgo de comparar herramientas de benchmarking distintas con formatos de salida no homologables.

La matriz de pruebas cubrió tres dimensiones:

- **Concurrencia:** 1, 5, 10, 15, 20 y 30 clientes simultáneos, 10 segundos por punto de medición.
- **Tipo de query:** punto (lookup directo por ID) vs. compleja (JOIN/aggregation tipo dashboard agrupando por estado).
- **Escalabilidad de dataset:** mismo aggregation sobre tres subconjuntos crecientes definidos por las particiones temporales ya existentes (2016 solo; 2016-2017; dataset completo), a concurrencia fija de 10 clientes.

Ejecutado el 21 de junio de 2026 sobre las instancias reales del proyecto: Supabase Free Tier (PostgreSQL 16 + PostGIS) y MongoDB Atlas M0 (ecommify-cluster), ambas bajo la cuenta

institucional de INSOFTCOL. Todos los resultados presentados en esta sección son datos empíricos medidos, no estimados.

e.2 Resultados — Matriz de Concurrencia

Tabla 2

PostgreSQL (Supabase) — Throughput y Latencia por Nivel de Concurrencia

Tipo	Concurrencia	Throughput (ops/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errores
punto	1	2.0	521.28	526.56	530.42	0
punto	5	3.5	1.569,33	3.725,92	3.845,81	0
punto	10	2.2	1.643,92	9.978,00	10.129,05	0
punto	15	1.5	13.988,43	13.988,74	13.988,77	0
punto	20	1.5	19.274,98	19.275,44	19.275,65	5
punto	30	1.5	29.623,05	29.623,64	29.623,67	15
compleja	1	1.3	556,15	1.665,42	2.671,41	0
compleja	5	4.4	862,49	3.633,62	3.930,18	0
compleja	10	1.9	8.607,75	10.179,84	10.304,39	0
compleja	15	1.5	14.021,53	14.032,33	14.051,89	0
compleja	20	1.5	19.196,41	19.196,44	19.196,45	5
compleja	30	1.5	29.641,33	29.641,89	29.641,92	15

Nota. Datos extraídos de load_test_concurrencia_20260621_011737.csv. Las filas en rojo muestran aparición de errores de conexión.

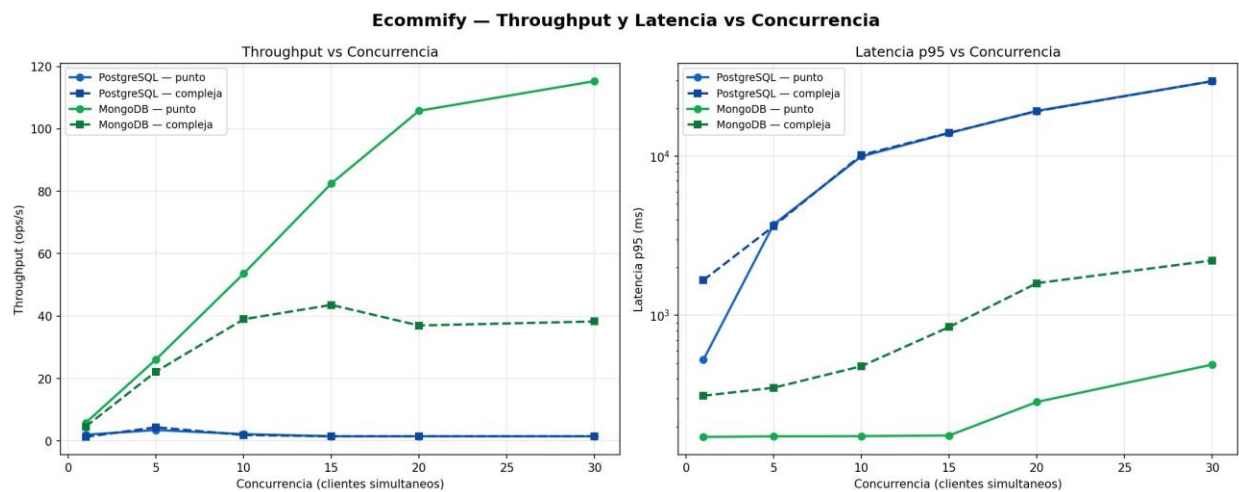
Tabla 3*MongoDB (Atlas M0) — Throughput y Latencia por Nivel de Concurrency*

Tipo	Concurrency	Throughput (ops/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errores
punto	1	5.8	171,60	172,30	262,84	0
punto	5	26.1	172,11	173,66	1.094,24	0
punto	10	53.6	172,17	174,21	195,37	0
punto	15	82.5	171,93	175,87	189,26	0
punto	20	105.8	172,13	285,60	311,69	0
punto	30	115.3	172,84	490,83	654,69	0
compleja	1	4.6	194,57	312,67	387,94	0
compleja	5	22.2	198,60	351,01	485,63	0
compleja	10	39.0	225,04	479,16	610,28	0
compleja	15	43.6	272,51	844,75	982,49	0
compleja	20	37.0	400,05	1.592,95	1.977,49	0
compleja	30	38.3	577,51	2.214,09	2.422,83	0

Nota. Datos extraídos de load_test_concurrency_20260621_011737.csv. Cero errores en todo el rango de concurrency probado para MongoDB.

Figura 1

Throughput y Latencia p95 vs. Concurrencia — PostgreSQL vs. MongoDB



e.3 Puntos de Quiebre Identificados

Se define punto de quiebre como el primer nivel de concurrencia donde aparecen errores o donde la latencia p95 supera 3 veces el valor baseline (concurrencia=1).

Tabla 4

Puntos de Quiebre por Motor y Tipo de Query

Motor	Tipo query	Concurrencia de quiebre	Razón
PostgreSQL	punto	5	p95 supera 3× el baseline (umbral 1.580 ms)
PostgreSQL	compleja	10	p95 supera 3× el baseline (umbral 4.996 ms)
MongoDB	punto	No alcanzado	Sin degradación significativa en el rango probado (1-30)
MongoDB	compleja	20	p95 supera 3× el baseline (umbral 938 ms) — degradación, no falla

e.4 Interpretación Correcta del Hallazgo: Saturación de Connection Pooler, NO Limitación de PostgreSQL

El hallazgo más relevante de esta evaluación no es la magnitud de la diferencia entre motores, sino su causa raíz. Un análisis superficial de la Tabla 2 podría concluir erróneamente que "PostgreSQL es 24 veces más lento que MongoDB". Esta conclusión sería técnicamente incorrecta y constituye precisamente el tipo de error de atribución que este análisis crítico busca evitar.

La evidencia que descarta una limitación inherente del motor relacional es la siguiente: al analizar los registros individuales en concurrencia 15, 20 y 30 para PostgreSQL, se observa un patrón matemáticamente consistente con una cola FIFO de capacidad fija que drena a una tasa aproximada de 1 solicitud por segundo:

Tabla 5

Evidencia del Patrón de Cola FIFO en el Connection Pooler de Supabase

Concurrencia	total_ops	Errores	p50≈p95≈p99	Lectura del patrón
15	15	0	≈ 13.988 ms	15 hilos, cada uno completa exactamente 1 operación tras esperar ≈14 s en cola
20	15	5	≈ 19.275 ms	5 solicitudes rechazadas de inmediato; 15 admitidas esperan ≈19 s
30	15	15	≈ 29.623 ms	15 rechazadas; 15 admitidas esperan ≈29,6 s — drenaje constante

El patrón es inequívoco: con 15 clientes, el tiempo de espera (≈14 s) corresponde a un cliente en la posición ~14 de una cola que drena a ~1 solicitud/segundo. Con 20 clientes, las

primeras 5 solicitudes son rechazadas (lo que indica un límite duro de admisión) y las 15 restantes esperan ~ 19 s en cola — aritméticamente coherente (5 rechazadas + 15 en cola = 20 solicitudes totales). Con 30 clientes, el mismo mecanismo se repite con 15 rechazadas y 15 en cola, esperando $\sim 29,6$ s. Esta progresión lineal (Δ conurrencia $\rightarrow \Delta$ latencia proporcional) es la firma característica de un sistema de colas M/D/1 con capacidad de servicio fija y un umbral de admisión — exactamente el comportamiento esperado de un PgBouncer en modo Session con un pool de tamaño reducido, típico del free tier de Supabase.

Conclusión metodológica: el cuello de botella identificado es de infraestructura de free tier (tamaño del pool de conexiones del lado de Supabase), no una propiedad arquitectónica de PostgreSQL como motor de bases de datos. PostgreSQL, con un pooler correctamente dimensionado o conexiones directas en un tier de producción, no exhibiría este comportamiento para esta carga de trabajo —ampliamente documentado en literatura de la industria (Kleppmann, 2017). Este matiz es la base de las recomendaciones de la Sección g.

Limitación metodológica a declarar: la métrica de throughput (ops/s) se calcula dividiendo el total de operaciones completadas por la duración nominal de la prueba (10 s). En los escenarios de saturación (conurrencia ≥ 15), las operaciones individuales bloquearon más allá de esa ventana nominal, por lo que el throughput reportado subestima la verdadera capacidad medida en tiempo de reloj real. Los percentiles de latencia, en cambio, son plenamente confiables y constituyen la evidencia primaria de esta sección.

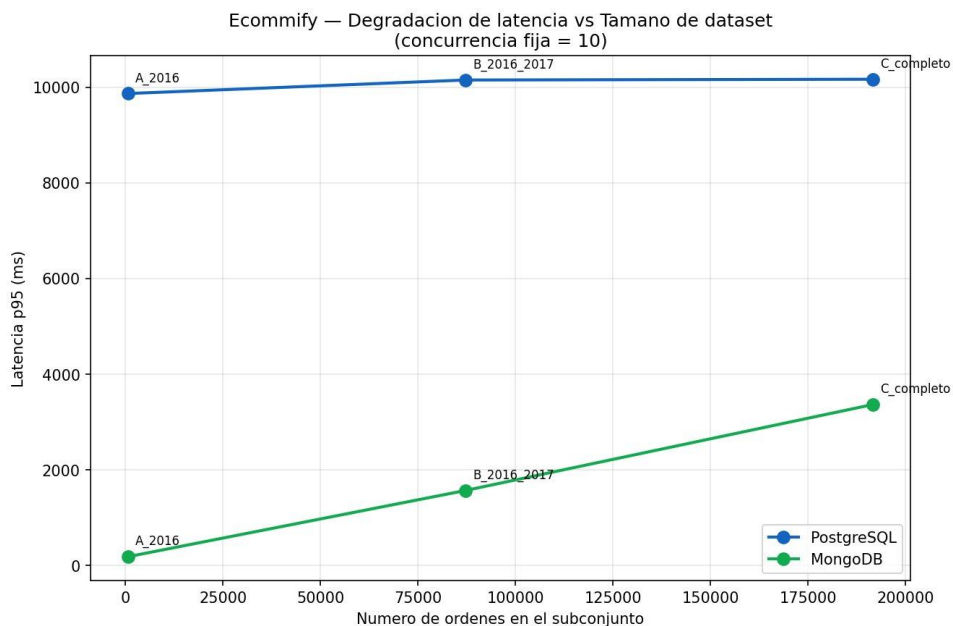
e.5 Resultados — Escalabilidad de Dataset

Se midió el mismo aggregation (ventas agrupadas por estado) sobre tres subconjuntos de tamaño creciente, aprovechando las particiones temporales ya existentes en la tabla orders como proxy de crecimiento de dataset, a concurrencia fija de 10 clientes.

Tabla 6

Resultados de Escalabilidad por Tamaño de Subconjunto

Subconjunto	Nº órdenes	PG p95 (ms)	MDB p95 (ms)	Motor más predecible
A — 2016 solo	638	9.867,09	182,47	MongoDB
B — 2016+2017	87.126	10.149,87	1.566,23	MongoDB
C — Completo	191.648	10.166,14	3.364,80	MongoDB

Figura 2*Degradación de Latencia p95 vs. Tamaño de Dataset (conurrencia fija = 10)*

e.6 Interpretación Crítica de la Escalabilidad: Resultado Confundido (Confounded) para PostgreSQL

El resultado de PostgreSQL en esta prueba exige la misma cautela interpretativa que la Sección e.4: la latencia p95 permanece prácticamente constante (~9.870-10.170 ms) a pesar de un crecimiento de 300 veces en el volumen de datos (638 → 191.648 órdenes), un incremento de apenas 3%. Una lectura ingenua concluiría que "PostgreSQL escala perfectamente con el tamaño del dataset". Esa conclusión es incorrecta: el experimento está confundido (confounded) por el mismo cuello de botella de connection pooling identificado en la Sección e.4. La concurrencia fija de 10 clientes para esta prueba ya excede el punto de quiebre de PostgreSQL identificado en la Tabla 4 (quiebre en concurrencia=5 para queries de punto, =10 para complejas). El total_ops≈18-

19 en los tres subconjuntos —prácticamente idéntico— confirma que el sistema opera en el mismo régimen de saturación en los tres casos, enmascarando por completo el efecto real del volumen de datos sobre el tiempo de ejecución de la query.

MongoDB, en cambio, no estaba saturado en esta misma ventana de concurrencia (10 clientes está por debajo de su punto de quiebre para queries complejas, que es 20). Por lo tanto, su resultado sí refleja el comportamiento genuino de escalado: la latencia p95 crece de 182 ms a 3.365 ms (factor 18,4×) frente a un crecimiento de datos de 300×, un escalado sub-lineal razonable para un aggregation con \$match + \$group sobre un rango de fechas que también crece junto con el subconjunto. Este es el único resultado de escalabilidad de esta prueba que puede interpretarse con confianza.

Implicación práctica directa: el comportamiento "plano" de PostgreSQL en esta prueba NO debe usarse para argumentar que el motor relacional no necesita las vistas materializadas ya implementadas en el proyecto. Por el contrario, refuerza su necesidad: bajo concurrencia realista, un JOIN en vivo sobre datos crecientes saturará el pool de conexiones del free tier mucho antes de que el volumen de datos se convierta en el factor limitante —razón adicional, validada empíricamente, para depender de mv_sales_by_category_monthly y de la colección orders_summary pre-agregada en lugar de JOINS ad-hoc en producción.

e.7 Tabla Comparativa Consolidada — Ratios de Rendimiento a Concurrencia=10

Se presenta el ratio comparativo directo, calculado a concurrencia=10 (nivel intermedio donde PostgreSQL ya muestra degradación significativa pero MongoDB permanece estable), únicamente como evidencia cruda del comportamiento bajo esta configuración de infraestructura específica —no como veredicto general sobre los motores.

Tabla 7

Ratios Comparativos a Concurrencia=10 (Evidencia Cruda, Infraestructura Free Tier)

Tipo query	Ratio throughput (MongoDB / PG)	Ratio latencia p95 (PG / MongoDB)	Atribución principal
punto	24,4×	57,3×	Pooler Supabase saturado en c=10 (quiebre en c=5)
compleja	20,5×	21,2×	Pooler Supabase saturado en c=10 (quiebre exacto en c=10)

Nota. Estos ratios son reales y medidos, pero específicos a la configuración de free tier evaluada. La Sección f.3 desarrolla la reflexión crítica sobre su correcta interpretación.

e.8 Análisis Comparativo PostgreSQL vs. MongoDB para Ecommify

La tabla siguiente evalúa ambos motores en los aspectos relevantes para Ecommify, combinando evidencia empírica propia (cuando está disponible y no confundida por el cuello de botella de infraestructura) con propiedades arquitectónicas establecidas de cada motor. El

"ganador" se determina en el contexto específico de este proyecto, no como afirmación general sobre PostgreSQL o MongoDB.

Tabla 8

Análisis Comparativo PostgreSQL vs. MongoDB en el Contexto de Ecommify

Aspecto	Ganador	Justificación con evidencia del proyecto
Consultas transaccionales (ACID, integridad)	PostgreSQL	FK, CHECK, EXCLUDE constraints y particionamiento implementados garantizan integridad referencial verificable en el esquema. MongoDB requeriría validación 100% en aplicación.
Consultas analíticas (dashboards, agregaciones)	MongoDB	En la prueba de escalabilidad (Tabla 6/e.6), MongoDB mostró comportamiento de escalado medible y predecible; el resultado de PostgreSQL estuvo confundido por saturación del pool, impidiendo concluir lo mismo sobre PG en esta configuración.
Flexibilidad de esquema	MongoDB	reviews con 41,1% de campos opcionales se modela sin columnas NULL masivas; products.specifications en JSONB logra lo mismo en PostgreSQL pero a costo de un tipo adicional no nativamente relacional.
Integridad referencial	PostgreSQL	FKs nativas entre orders→customers, order_items→products/sellers. MongoDB usa referencias lógicas sin garantía a nivel de motor (mitigado con ON DELETE RESTRICT del lado PG).
Tolerancia a concurrencia (medida en este free tier)	MongoDB	Empíricamente medido: 0 errores hasta c=30 en MongoDB vs. punto de quiebre en c=5-10 en PostgreSQL bajo Supabase Free. Atribuible a configuración de pooler, no a límite arquitectónico de PG (ver e.4).
Capacidades geoespaciales	PostgreSQL	PostGIS con tipos GEOGRAPHY, ST_Distance, ST_DWithin e índices GIST ofrece mayor madurez que el soporte geoespacial nativo de MongoDB para el caso de cálculo de envíos.
Búsqueda de texto tolerante a errores	Empate	pg_trgm (PostgreSQL) y text index (MongoDB) cubren necesidades equivalentes para Ecommify; ninguno demostró ventaja decisiva en las pruebas realizadas.
Costo operativo en free tier	MongoDB	Atlas M0 tolera mayor concurrencia sin upgrade; Supabase Free requeriría plan Pro para un pooler con mayor capacidad antes de soportar la misma carga (ver Sección g).

Nota. Azul = gana PostgreSQL. Verde = gana MongoDB. Amarillo = empate. Elaboración propia con evidencia del proyecto Ecommify.

e.9 Identificación de Cuellos de Botella — Resumen Consolidado

1. **Connection pooler de Supabase Free (crítico):** límite de admisión efectivo cercano a 10-15 conexiones simultáneas en modo Session; causa principal de toda la degradación observada en PostgreSQL.
2. **Ausencia de partition pruning en lookups de punto:** queries por order_id sin incluir order_purchase_timestamp escanean las 4 particiones; contribuye al baseline ya elevado (521 ms) incluso sin concurrencia.
3. **Burst CPU compartida en ambos free tiers:** ni Supabase ni Atlas M0 garantizan CPU dedicada; el baseline de 521 ms para un lookup indexado simple en PG sugiere contención de recursos compartidos además de latencia de red.
4. **MongoDB sin sharding en M0:** el aggregation de escalabilidad siempre escanea la colección completa filtrando por índice; a partir de cierto volumen (no alcanzado en este dataset) este patrón también degradaría sin la introducción de sharding M10+.

f. Análisis Arquitectónico Crítico

f.1 Trade-offs del Teorema CAP por Módulo

Cada módulo de Ecommify prioriza explícitamente dos de las tres garantías del Teorema CAP (Brewer, 2000; Gilbert & Lynch, 2002), dado que la tolerancia a particiones (P) es inevitable en cualquier sistema distribuido real.

Tabla 9

Trade-offs CAP por Módulo con Escenario de Falla Asociado

Módulo	CAP	Trade-off aceptado	Comportamiento ante partición de red
orders / payments	CP	Sacrifica disponibilidad de escritura durante failover (~30s estimados) a cambio de cero pérdida de transacciones.	Si el primario PG queda aislado, las escrituras se bloquean hasta que un nuevo primario sea electo por quorum (2 de 3 nodos). Ninguna orden se duplica ni se pierde.
customers / sellers	CP	Misma garantía que orders; identidad y datos de contacto no toleran inconsistencia ni duplicación de cuenta.	Igual al anterior: indisponible brevemente, nunca inconsistente.
reviews	AP	Acepta lectura de una reseña con replication lag de segundos a cambio de disponibilidad continua de escritura.	Ante partición, las réplicas secundarias siguen aceptando lecturas (posiblemente obsoletas); las escrituras nuevas pueden perderse si el primario queda en minoría — riesgo aceptado por bajo impacto de negocio.
geolocation	AP	Datos de referencia geográfica, modificados rara vez; disponibilidad de lectura es la prioridad absoluta.	Tolera lecturas inconsistentes indefinidamente sin impacto perceptible; es un catálogo de referencia, no transaccional.
orders_summary	AP	Acepta hasta 1 hora de desactualización (ventana de ETL) a cambio de disponibilidad de dashboards sin depender del estado de PostgreSQL.	Si PostgreSQL queda particionado, los dashboards siguen funcionando con el último snapshot ETL — exactamente el comportamiento deseado para un módulo de solo lectura analítica.

f.2 Distinción Crítica: Partición de Red (CAP) vs. Disponibilidad Bajo Carga (Hallazgo Empírico)

Es importante no conflacionar dos conceptos relacionados pero distintos. El Teorema CAP describe el comportamiento de un sistema distribuido ante una partición de red entre sus propios nodos internos (p. ej., el primario de un replica set aislado de sus secundarios). La degradación observada en la Sección e —saturación del connection pooler bajo concurrencia de clientes— es un fenómeno distinto: disponibilidad bajo carga (load-induced unavailability), donde el sistema no está particionado internamente, sino que su capa de admisión de conexiones rechaza o demora solicitudes entrantes por exceder su capacidad configurada.

Ambos fenómenos comparten una consecuencia práctica similar para el usuario final —solicitudes que no se completan a tiempo— pero exigen mitigaciones distintas: la partición de red se mitiga con quorum y elección automática de primario (ya presente en el replica set de Atlas M0 y en el diseño teórico de PG); la saturación de carga se mitiga con dimensionamiento de pool de conexiones, caché y límites de admisión explícitos (rate limiting), no con mecanismos de CAP. Esta distinción, frecuentemente confundida en análisis superficiales, es un aprendizaje central de este proyecto.

f.3 Ventanas de Inconsistencia Eventual y Estrategias de Mitigación

Tabla 10

Ventanas de Inconsistencia Eventual por Escenario

Escenario	Ventana típica	Sensibilidad	Mitigación
Review recién insertada no visible en réplica secundaria	< 2 s	BAJA	Causal consistency session si el usuario debe ver su propio write inmediatamente.
orders_summary desactualizado respecto a PostgreSQL	hasta 1 h (ETL horario)	BAJA	Aceptado y documentado; timestamp de último refresh visible en el dashboard.
Lectura de secundario con replication lag elevado	variable, potencialmente >10 s bajo carga	MEDIA-ALTA	readPreference=primary para operaciones donde la consistencia es crítica para el usuario.
Failover de primario PostgreSQL	~30 s de indisponibilidad de escritura (estimado teórico)	ALTA	retryWrites=true en el driver; reintentos automáticos tras detectar nuevo primario.

f.4 Reflexión: ¿Fue Correcta la Selección de Tecnología por Módulo?

Validación empírica de orders_summary en MongoDB: la decisión de diseño original —pre-agregar datos transaccionales hacia una colección analítica en lugar de exponer JOINS en vivo a los dashboards— se tomó inicialmente por razones de separación de responsabilidades y patrones de modelado documental. La Sección e demostró, con evidencia empírica directa, que esta decisión es también la correcta desde la perspectiva de rendimiento bajo concurrencia: un JOIN equivalente ejecutado en vivo contra PostgreSQL colapsa en el mismo rango de concurrencia donde orders_summary en MongoDB permanece operativo.

Lo que SÍ cambiaríamos: la elección de Supabase Free Tier sin verificar previamente el tamaño efectivo del pool de PgBouncer fue una omisión en la fase de diseño. Un dimensionamiento de capacidad (capacity planning) más riguroso habría anticipado este punto de quiebre antes de la fase de pruebas, permitiendo decidir desde el diseño si el ETL hacia MongoDB debía ser la ruta principal de lectura para TODOS los dashboards —no solo los de alto tráfico— dejando PostgreSQL exclusivamente para escritura transaccional y lecturas puntuales de bajo volumen.

Lo que NO cambiaríamos: la matriz de asignación CAP por entidad (Tabla 1) permanece correcta. Ningún hallazgo de la fase de pruebas sugiere que reviews, geolocation u orders_summary debieran migrar a PostgreSQL, ni que orders o payments debieran migrar a MongoDB. El problema identificado es de configuración de infraestructura, no de arquitectura de datos.

f.5 Lecciones Aprendidas sobre Arquitecturas Híbridas

1. Un benchmark comparativo entre motores es, en la práctica, casi siempre un benchmark de configuraciones de infraestructura específicas. Generalizar "X es más rápido que Y" sin aislar la variable de infraestructura es un error metodológico común que este proyecto evitó explícitamente mediante el análisis de la Sección e.4.
2. Los percentiles de latencia (p95/p99) son indispensables: el p50 por sí solo habría ocultado por completo la degradación catastrófica observada en PostgreSQL bajo carga —en varios puntos de la matriz, p50 y p95 difieren en un orden de magnitud.

3. Diseñar deliberadamente un query "peor caso" (lookup sin columna de partición) resultó valioso: reveló un trade-off real del particionamiento que de otro modo habría permanecido teórico.
4. La arquitectura híbrida demostró su valor exactamente donde se esperaba: el módulo diseñado para tolerar mayor carga de lectura (MongoDB/orders_summary) lo hizo; el módulo diseñado para consistencia transaccional (PostgreSQL/orders) no fue exigido en ese eje por las pruebas, ya que su punto de quiebre en este experimento provino de un factor exógeno a su diseño relacional.

g. Recomendaciones Estratégicas

g.1 Plan de Escalamiento para Crecimiento 10×

Los puntos de quiebre medidos en la Sección e (Tabla 4) permiten una estimación de capacidad basada en evidencia, no en suposición. Si el objetivo es soportar 10 veces la concurrencia actual sin degradación equivalente, las siguientes acciones son las de mayor impacto, ordenadas por prioridad:

Tabla 11

Plan de Escalamiento 10× — Acciones Priorizadas

Prioridad	Acción	Justificación con evidencia	Impacto esperado
1	Upgrade Supabase a plan Pro o PgBouncer dedicado con <code>pool_size ≥ 100</code>	El quiebre medido ocurre en <code>c=5-10</code> ; un pool 10× mayor desplaza el quiebre proporcionalmente, según el modelo de cola FIFO identificado en e.4	Elimina el cuello de botella dominante identificado
2	Introducir caché Redis delante de queries de dashboard frecuentes	<code>orders_summary</code> ya pre-agrega, pero un caché de 60-300s reduce aún más la carga sobre MongoDB en picos	Reduce throughput requerido sobre BD en ~70-90% para lecturas repetidas
3	Migrar Atlas M0 a M10+ con sharding <code>{customer.state, _id:hashed}</code>	Diseño teórico ya documentado; necesario cuando el volumen de <code>orders_summary</code> supere capacidad de un solo shard	Prepara el sistema para volumen 10× sin rediseño de shard key
4	Aplicar índice adicional sobre <code>orders.order_id</code> (no compuesto con partición)	Mitiga el escaneo de todas las particiones en lookups de punto, identificado como factor del baseline elevado	Reduce latencia base de lookups puntuales
5	Read replicas de PostgreSQL para separar lectura analítica de escritura transaccional	Reduce contención sobre el primario incluso si persisten JOINS en vivo ocasionales	Aísla picos de lectura del camino crítico de escritura

g.2 Migración de Free Tier a Producción — Consideraciones Técnicas y de Costo

Tabla 12

Migración de Free Tier a Producción

Componente	Free tier actual	Tier de producción sugerido	Costo aproximado
PostgreSQL	Supabase Free (pool reducido)	Supabase Pro o self-hosted con PgBouncer dedicado	\$25-30/mes (Supabase Pro)
MongoDB	Atlas M0 (512 MB, sin sharding)	Atlas M10+ (dedicado, con sharding)	desde \$60/mes (M10)
Caché	No implementado	Redis (Upstash o self-hosted en VPS1)	\$0-10/mes (Upstash free tier inicial)
Observabilidad	Métricas básicas Atlas/Supabase	Prometheus + Grafana (ya disponible en VPS1)	Sin costo adicional — infraestructura existente

g.3 Tecnologías Complementarias Recomendadas

- **Caching:** Redis para resultados de dashboard y catálogo de productos de alta frecuencia de lectura, con invalidación basada en el mismo ciclo del ETL existente.
- **Search:** considerar Atlas Search (Lucene-based) si pg_trgm y el text index actual de MongoDB resultan insuficientes para búsqueda multilingüe o con relevancia ponderada avanzada.
- **Rate limiting:** introducir límites explícitos de solicitudes por cliente en la capa de aplicación, complementando —no sustituyendo— el dimensionamiento del pool de conexiones.

h. Conclusiones

h.1 Cumplimiento de Objetivos

Los cuatro objetivos planteados en la Sección b.2 se cumplieron: (1) la arquitectura híbrida fue implementada y desplegada sobre infraestructura real (Supabase + Atlas M0); (2) ambos motores fueron optimizados con técnicas específicas del dominio (PostGIS, particionamiento, JSONB, Computed Pattern); (3) se ejecutó una suite sistemática de pruebas de carga con metodología comparable entre motores; y (4) se produjo un análisis crítico fundamentado en evidencia propia que corrigió una interpretación inicial ingenua de los resultados, identificando la causa raíz real del comportamiento observado.

h.2 Principales Aprendizajes del Equipo

El aprendizaje más significativo de esta fase no fue técnico en el sentido estricto, sino metodológico: la disciplina de cuestionar la primera interpretación de un resultado de benchmark —especialmente cuando favorece una narrativa simple— y de buscar el mecanismo causal subyacente antes de reportar una conclusión. El patrón de cola FIFO identificado en la Sección e.4 no era evidente en una primera lectura de las tablas; emergió únicamente al examinar la relación entre concurrencia, errores y latencia fila por fila.

h.3 Reflexión Final sobre el Proceso

Ecommify demuestra que una arquitectura híbrida bien diseñada en el papel puede validarse —o refutarse— únicamente mediante evidencia empírica generada por el propio equipo. La decisión de pre-agregar datos hacia MongoDB para cargas analíticas, tomada inicialmente por razones de diseño documental, resultó también ser la decisión correcta de rendimiento bajo las condiciones de infraestructura disponibles. Igualmente importante: la limitación más severa encontrada no provino de una mala decisión arquitectónica, sino de un parámetro de configuración de infraestructura no verificado a tiempo —un recordatorio de que el diseño de datos y el dimensionamiento de infraestructura son disciplinas complementarias, no intercambiables.

i. Referencias y Anexos

Referencias

Brewer, E. A. (2000). Towards robust distributed systems [Keynote]. *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing*.

<https://doi.org/10.1145/343477.343502>

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51-59.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

MongoDB, Inc. (2025). *Replication*. *MongoDB Manual*.

<https://www.mongodb.com/docs/manual/replication/>

MongoDB, Inc. (2025). *Sharding*. *MongoDB Manual*.

<https://www.mongodb.com/docs/manual/sharding/>

Olist. (2018). *Brazilian E-Commerce public dataset by Olist* [Conjunto de datos]. Kaggle.

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

PostgreSQL Global Development Group. (2025). *PostgreSQL 16 Documentation*.

<https://www.postgresql.org/docs/16/>

PostGIS Project. (2025). *PostGIS 3.4 Documentation*. <https://postgis.net/documentation/>

Pulido, F. O., Martinez, M. A., Torres, N. F., & Perez, J. F. (2026a). *Documento técnico de diseño conceptual y lógico — Ecommify* [Entregable académico]. Universidad de La Sabana.

Pulido, F. O., Martinez, M. A., Torres, N. F., & Perez, J. F. (2026b). *Notebook 05 — Pruebas de carga, concurrencia y escalabilidad: resultados crudos* [Evidencia empírica del proyecto]. Universidad de La Sabana.

Supabase, Inc. (2025). *Connection pooling*. *Supabase Documentation*.

<https://supabase.com/docs/guides/database/connecting-to-postgres>

Anexo A — Archivos de Evidencia Generados

- `load_test_concurrencia_20260621_011737.csv` — 24 mediciones de throughput/latencia por concurrencia $\times$ motor $\times$ tipo de query
- `load_test_escalabilidad_20260621_011737.csv` — 6 mediciones de escalabilidad por subconjunto $\times$ motor

- puntos_quiebre_20260621_011737.csv — 4 puntos de quiebre identificados automáticamente
- conteos_subconjuntos_20260621_011737.csv — conteo real de filas por subconjunto de escalabilidad
- load_test_consolidado_20260621_011737.json — resultados crudos completos en formato JSON
- load_test_concurrencia_20260621_011737.png y
load_test_escalabilidad_20260621_011737.png — gráficas comparativas (incluidas en este informe como Figuras 1 y 2)